Database development with PL/SQL INSY 8311

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | | eric.maniraguha@auca.ac.rw | LinkedIn Profile

6h00 pm – 9h50 pm

- Monday A -G207
- Tuesday B-G204
- Wednesday E-106
- Thursday F-G307

March 2025

Database development with PL/SQL

Reference reading

- Difference between Cursor and Trigger in DBMS
- Difference between Cursor and Trigger in PLSQL
- YouTube Tutorial: PL/SQL tutorial 26: Introduction to PL/SQL Cursor in Oracle Database By Manish Sharma

Lecture 08 – PL/SQL Cursors in Oracle Database

The objectives of cursors and triggers in PL/SQL

What is a Cursor in PL/SQL?

A **cursor** in PL/SQL is a **pointer to a context area** where the result set of a query is stored. **It allows row-by-row processing of query results instead of handling all rows at once**, making it useful for situations where you need to process records individually.

Why Use Cursors?

Row-by-Row Processing

- Unlike regular SQL queries that process all rows at once, a cursor allows you to **process one row at a time**.

Handling Multiple Rows in PL/SQL

- PL/SQL does not support direct row-by-row iteration over a query result, so cursors help to **retrieve and manipulate each row** separately.

Efficient Data Handling

- Useful when performing **complex operations** on each row before moving to the next (e.g., calculations, updates, or inserting into another table).

Improved Control over Query Execution

- Provides better control with actions like **OPEN, FETCH, and CLOSE**, ensuring resource management and structured data access.

Encapsulation of Query Logic

- An explicit cursor allows you to define a **named result set**, making it easier to reuse and maintain query logic.

Introduction to Cursors in PL/SQL

Introduction to Cursors in PL/SQL

- When executing a SQL statement, especially SELECT, PL/SQL retrieves multiple rows. However, PL/SQL does not directly process multiple rows like SQL.
- A **cursor** allows row-by-row processing of a query result set.
- Cursors help in improving performance and handling large datasets efficiently.

Where to Apply Cursors in PL/SQL:

- **Implicit cursors**: Best for single-row queries where PL/SQL automatically manages the cursor.
- **Explicit cursors**: Use for multi-row queries when you need precise control over fetching and processing.
- **Cursor FOR loops**: Ideal for simplified row-by-row processing without manual OPEN, FETCH, and CLOSE operations.
- **REF cursors**: Suitable for dynamic query execution when the query structure is not fixed at compile time.

Types of Cursors

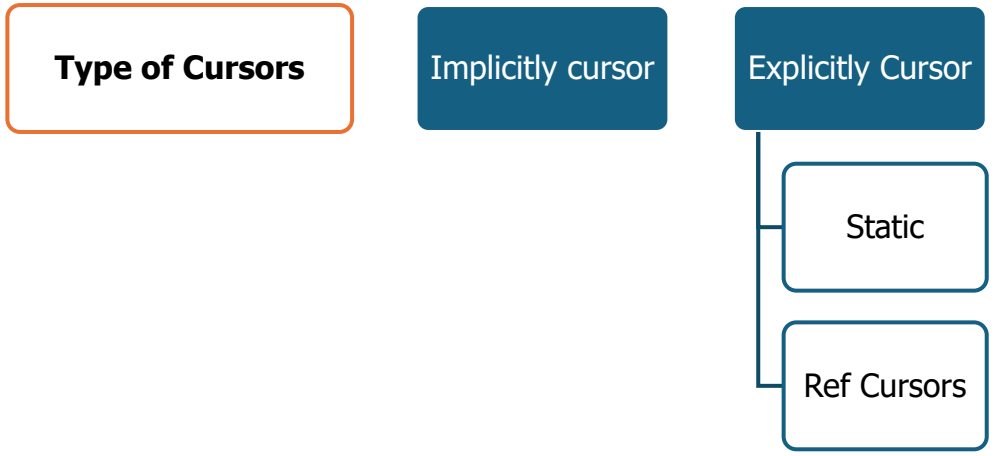
Implicit Cursor

- Created automatically when a SQL statement (like INSERT, UPDATE, DELETE, SELECT INTO) is executed.
- No explicit declaration or handling is required.

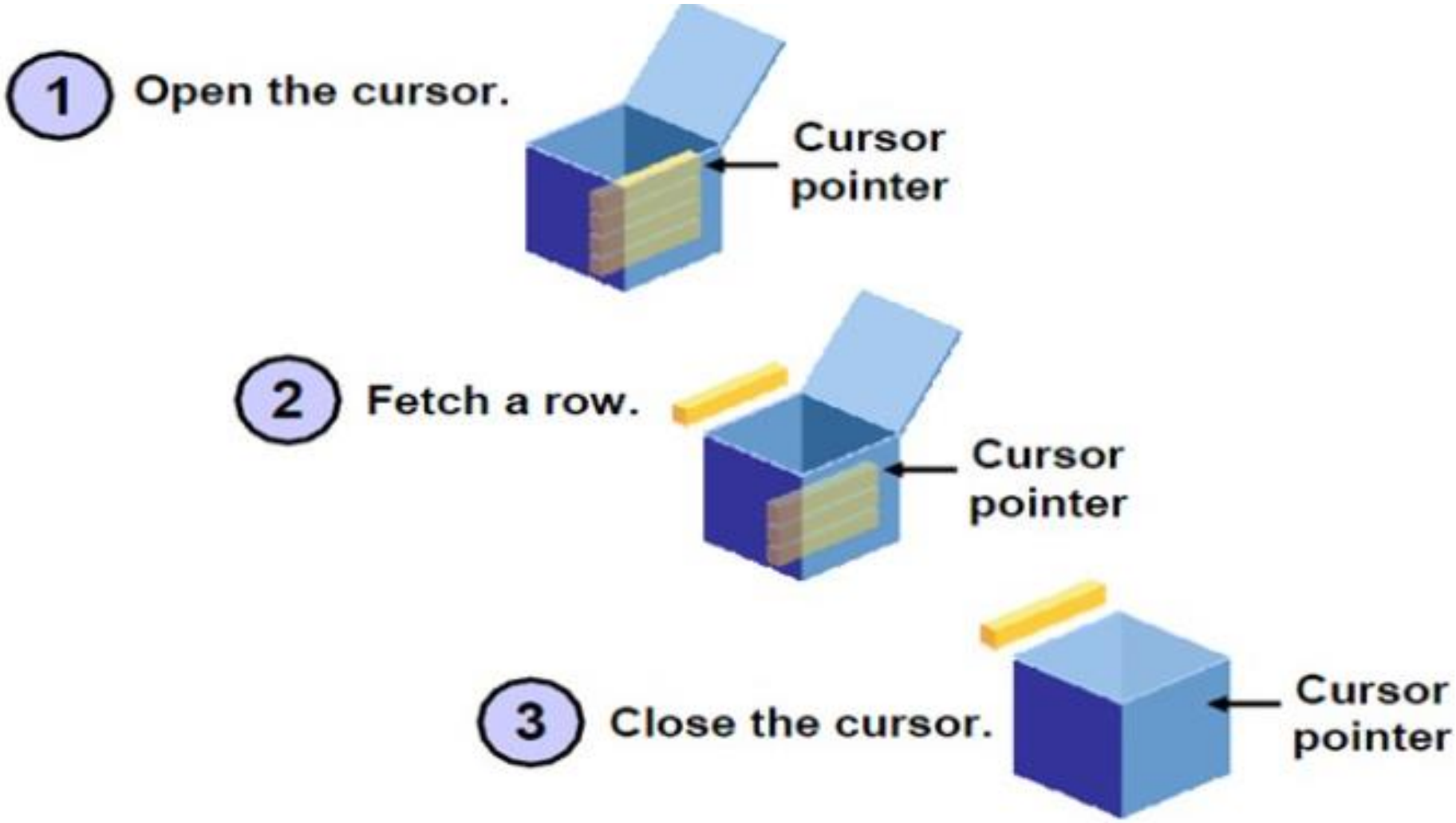
Explicit Cursor

- Declared explicitly in PL/SQL for handling multiple rows.
- Allows row-by-row processing.
- **Types of Explicit Cursors:**
  - **Static Cursor**: A predefined query that does not change at runtime.
  - **REF Cursor (Dynamic Cursor)**: A cursor variable that allows query assignment at runtime, making it dynamic.

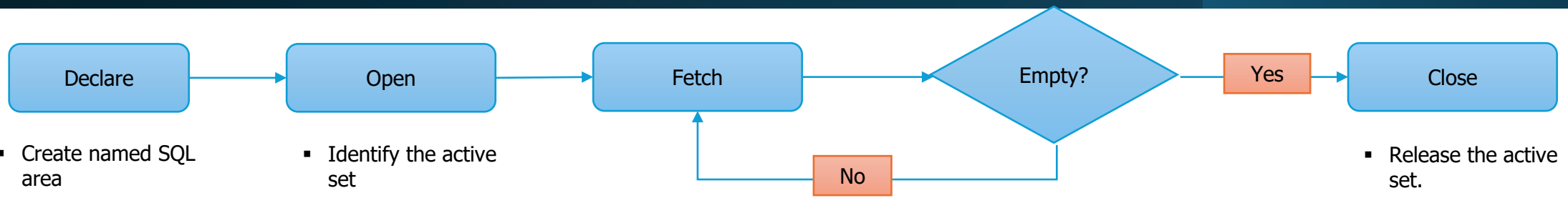
SQL*Plus



Controlling Explicit Cursor



Cursor Implementation with PL/SQL



Declare:
- A cursor named emp_cursor is declared to execute a SELECT statement that retrieves emp_id and emp_name from the employees table.
- Two variables, v_id and v_name, are declared to store the fetched values of emp_id and emp_name respectively.

Open:
- The cursor emp_cursor is opened, establishing a connection to the database and preparing it to fetch rows.

Loop:
- The program enters a loop to fetch rows one by one from the cursor.

Fetch:
- Each iteration of the loop retrieves the next row from the cursor and stores the values of emp_id and emp_name into the declared variables v_id and v_name.

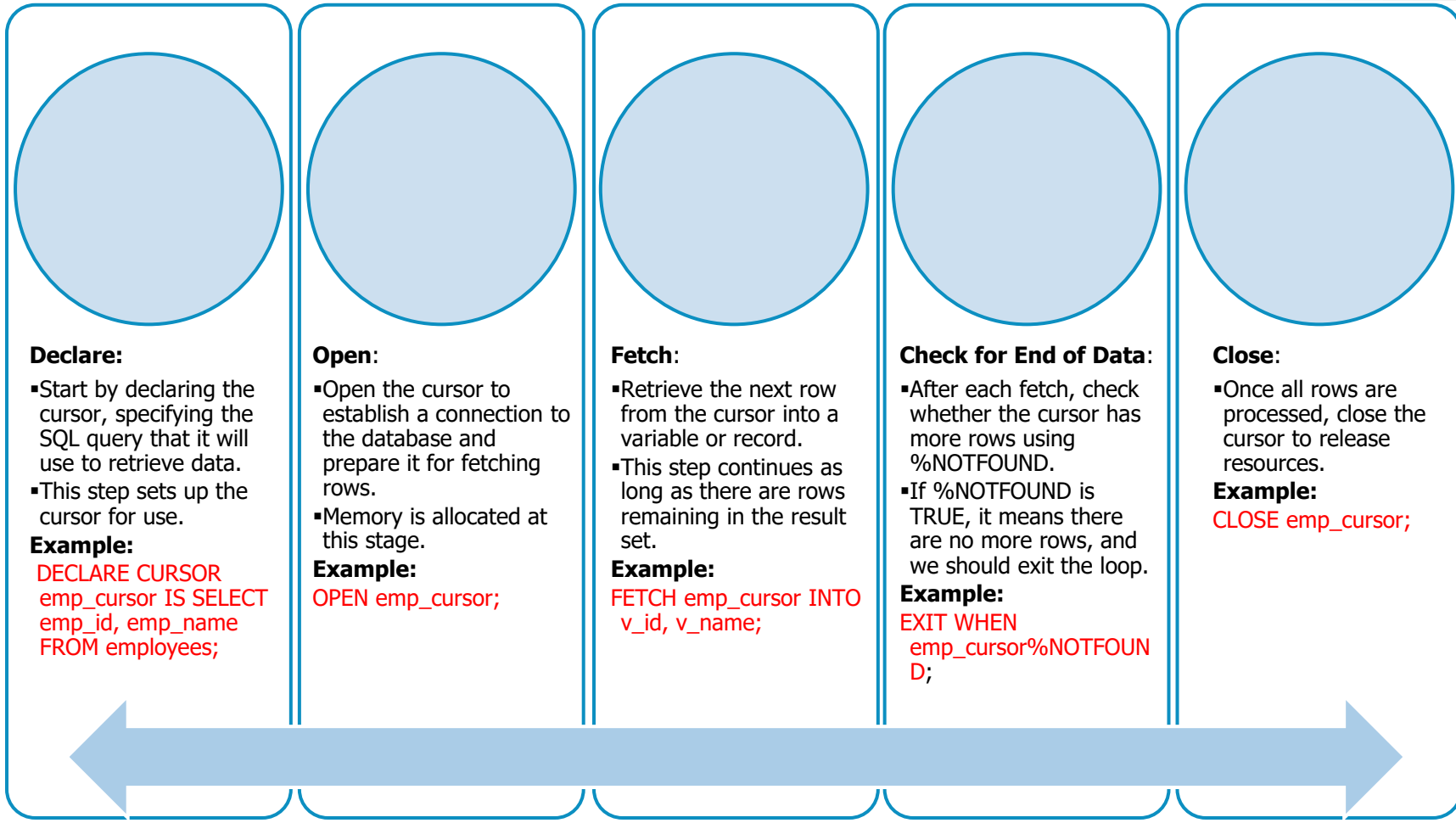
Check for End of Data:
- After each fetch, the program checks if there are no more rows in the cursor using the condition %NOTFOUND.
- If no more rows are found, the loop exits.

Output:
- If rows are still available, the program outputs the emp_id and emp_name for the current row using DBMS_OUTPUT.PUT_LINE.

Close:
- After processing all rows, the cursor is closed to release any resources associated with it and end the cursor's lifecycle.

Explicitly Cursors Example

```
DECLARE
    -- Declare the cursor to select emp_id and emp_name from the employees table
    CURSOR emp_cursor IS SELECT emp_id, emp_name FROM employees;

    -- Declare variables to store the fetched emp_id and emp_name
    v_id employees.emp_id%TYPE;
    v_name employees.emp_name%TYPE;

BEGIN
    -- Open the cursor to establish a connection and start fetching rows
    OPEN emp_cursor;

    -- Loop through each row fetched from the cursor
    LOOP
        -- Fetch the next row from the cursor and store the values in v_id and v_name
        FETCH emp_cursor INTO v_id, v_name;

        -- Exit the loop when no more rows are available in the cursor
        EXIT WHEN emp_cursor%NOTFOUND;

        -- Output the emp_id and emp_name for the current row
        DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ', Name: ' || v_name);
    END LOOP;

    -- Close the cursor after all rows have been fetched to release resources
    CLOSE emp_cursor;

END;
```



Cursor Applications in PL/SQL

Processing Multiple Rows from a Query
Cursors allow processing of multi-row results returned by SQL queries.
Example: Retrieving all employees from a department and applying logic to each employee's record (e.g., salary adjustments, status updates).

Implementing Complex Business Logic
Cursors are essential when applying complex business logic to each row individually.
Example: Calculating discounts, updating stock levels, or generating invoices one at a time.

Handling Large Result Sets
Cursors manage large result sets by fetching rows one at a time, thus avoiding memory overload.
Example: Fetching records from a large database and processing them row by row to prevent memory issues.

Dynamic SQL Execution (REF Cursors)
REF Cursors are used for dynamic SQL queries where the query structure is not known until runtime.
Example: Executing SQL queries passed as parameters or fetching data based on user input.

Iterating Over Complex Data Structures
Cursors are ideal for processing nested or complex data structures, such as results from joins, subqueries, or multi-table queries.
Example: Joining employee and department tables to display department-wise salary details.

Efficient Data Updates or Deletes
Cursors are used when rows need to be updated or deleted based on query results.
Example: Updating customer order statuses or deleting expired records.

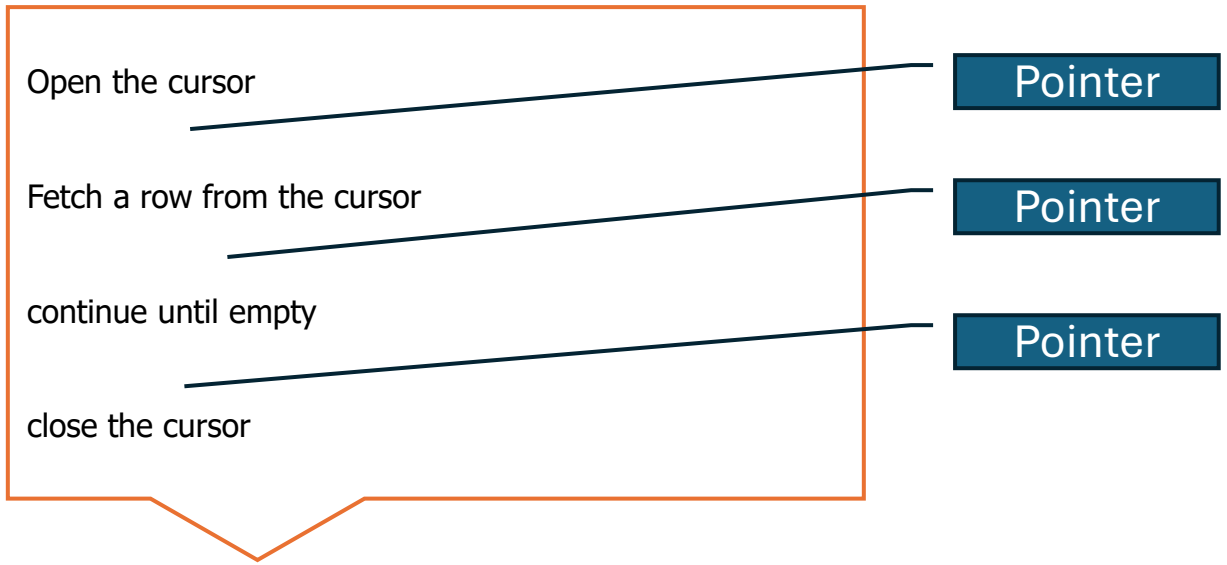
Data Transformation and Reporting
Cursors are useful when transforming data before outputting or storing it.
Example: Fetching sales data, transforming it into a specific format, and generating custom reports.

Looping Through Records in Bulk Operations
Cursors are helpful in bulk operations, such as batch jobs that need to update or insert multiple rows.
Example: Processing a batch of user account records for approval or activating/deactivating accounts.

Handling Multiple Related Tables
Cursors efficiently process data that spans multiple related tables.
Example: Fetching orders from an order table and iterating over related records in an order details table to update inventory.

Simplified Use with Cursor FOR Loops
The cursor FOR loop simplifies the row-by-row processing of a result set by automatically opening, fetching, and closing the cursor.
Example: Iterating over employee records to send email notifications about upcoming performance reviews.

Explicitly Cursor Syntax



Explicit Cursor (Basic Programming Structure)

- **Open the Cursor**: Initializes the cursor and sets it to the first row in the result set.
- **Fetch a Row**: Retrieves the current row and advances the cursor to the next row.
- **Continue Until Empty**: Repeat fetching until no more rows are left.
- **Close the Cursor**: Releases the cursor, freeing up resources.

```
DECLARE
    -- Step 1: Declare the cursor with a SELECT statement
    CURSOR cursor_name IS
        SELECT column1, column2, ...
        FROM table_name
        WHERE condition;

    -- Variables to hold the data from each row
    var1 table_name.column1%TYPE;
    var2 table_name.column2%TYPE;
    -- Add more variables as needed

BEGIN
    -- Step 2: Open the cursor
    OPEN cursor_name;

    -- Step 3: Fetch rows from the cursor
    LOOP
        FETCH cursor_name INTO var1, var2;

        -- Exit the loop when no more rows are found
        EXIT WHEN cursor_name%NOTFOUND;

        -- Process each row (e.g., display values or perform calculations)
        DBMS_OUTPUT.PUT_LINE('Column1: ' || var1 || ', Column2: ' || var2);
    END LOOP;

    -- Step 4: Close the cursor
    CLOSE cursor_name;
END;
/
```

SQL*Plus

Implicit cursor example and its purpose

Example	SQL Statement	Explanation
Update Statement	UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'IT';	Increases the salary of all employees in the IT department by 10%. An implicit cursor is created automatically, which finds and updates these rows based on the WHERE clause, then closes automatically.

Select Into Statement	DECLARE total_employees NUMBER;
	BEGIN
	SELECT COUNT(*) INTO total_employees FROM Employees;
	END;
	/
		Retrieves the total count of rows in the Employees table and stores it in the total_employees variable. An implicit cursor is used to handle the retrieval operation and closes right after execution.

Insert Statement	INSERT INTO Orders (OrderID, CustomerID, OrderDate) VALUES (123, 'C001', SYSDATE);	Inserts a new row into the Orders table with values for OrderID, CustomerID, and OrderDate. The implicit cursor processes this single-row insertion and closes automatically after the operation.
Delete Statement	DELETE FROM Customers WHERE Country = 'USA';	Deletes all rows in the Customers table where the Country is 'USA'. The implicit cursor locates these rows, performs the deletion, and then closes automatically.

Explanation of Implicit Cursors in Oracle PL/SQL

- **Implicit Cursors**: In Oracle PL/SQL, implicit cursors are created automatically by the database whenever a DML (Data Manipulation Language) operation like UPDATE, INSERT, DELETE, or a SELECT INTO query is executed.
- **Automatic Management**: Oracle opens, processes, and closes these cursors internally without user intervention, which simplifies handling of single-row or straightforward multi-row operations.
- **Efficiency**: Implicit cursors are generally more efficient than explicit cursors because they do not require explicit declaration, opening, fetching, or closing by the user.

Description of each cursor attributes

Cursor attributes provide crucial information about the status and behavior of cursors during execution. These attributes help you determine whether rows were fetched, how many rows were processed, and whether the cursor is still open.

%FOUND

Returns TRUE if the most recent FETCH operation successfully retrieved a row. Returns FALSE if no row was fetched. For **implicit cursors**, %FOUND returns TRUE if a DML (Data Manipulation Language) operation affected at least one row.

%NOTFOUND

Returns TRUE if the most recent FETCH did **not** retrieve a row (i.e., the end of the result set has been reached). Returns FALSE if a row was successfully fetched. This is the **opposite** of %FOUND.

%ISOPEN

Returns TRUE if the cursor is currently **open**. This is only applicable to **explicit cursors**, since **implicit cursors are automatically opened and closed by Oracle**, so %ISOPEN always returns FALSE for them.

%ROWCOUNT

Returns the **number of rows processed** by the cursor:

- For **implicit cursors**, it gives the number of rows affected by a DML operation.
- For **explicit cursors**, it shows the number of rows fetched **so far** since the cursor was opened.

Implicit Cursor:

1. SQL%OPEN
2. SQL%FOUND
3. SQL%NOTFOUND

1. SQL%ISOPEN
Explanation:
SQL%ISOPEN checks if an **implicit cursor** is currently open. However, for implicit cursors, this attribute **always returns FALSE** because Oracle **automatically opens and closes** them after the execution of a SQL statement.

Example:
After executing a DELETE statement on the ALLOWANCES table, SQL%ISOPEN will immediately return FALSE since the implicit cursor is already closed.

2. SQL%FOUND and SQL%NOTFOUND
Explanation:
These attributes are used to determine whether a **DML statement** (INSERT, UPDATE, DELETE) or SELECT INTO successfully affected or returned any rows.

- SQL%FOUND: Returns TRUE if **one or more rows** were affected.
- SQL%NOTFOUND: Returns TRUE if **no rows** were affected.

SQL%FOUND in a Procedure

```
CREATE OR REPLACE PROCEDURE delete_allowance(p_allowance_id NUMBER) AS
BEGIN
    -- Attempt to delete a row from the ALLOWANCES table with the specified ALLOWANCE_ID.
    DELETE FROM ALLOWANCES WHERE ALLOWANCE_ID = p_allowance_id;

    -- Check if any row was deleted
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Allowance with ID ' || p_allowance_id || ' deleted successfully.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No allowance found with ID ' || p_allowance_id || '.');
    END IF;

    -- Show SQL%ISOPEN (will be FALSE for implicit cursors)
    IF SQL%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor is still open.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Implicit cursor is already closed.');
    END IF;
END;
/
```

```
BEGIN
    delete_allowance(101);  -- Replace 101 with the actual allowance ID you want to
delete
END;
/
```

Implicit Cursor:

1. SQL%FOUND,
2. SQL%ROWCOUNT

Implicit Cursor

```
SET SERVEROUTPUT ON SIZE 100000;

CREATE OR REPLACE PROCEDURE delete_allowance(p_allowance_id NUMBER) AS
BEGIN
    -- Attempt to delete a row from the ALLOWANCES table with a specific ALLOWANCE_ID.
    DELETE FROM ALLOWANCES WHERE ALLOWANCE_ID = p_allowance_id;

    -- Use SQL attributes to check the outcome of the DELETE operation
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Allowance with ID ' || p_allowance_id || ' deleted successfully.');
        DBMS_OUTPUT.PUT_LINE('Rows deleted: ' || SQL%ROWCOUNT);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No allowance found with ID ' || p_allowance_id);
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data found for the given ID.');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Multiple rows found for the given ID.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

```
BEGIN
    -- Call the procedure with an example ID
    delete_allowance(101);  -- Replace 101 with the actual ALLOWANCE_ID you want to delete

EXCEPTION
    WHEN OTHERS THEN
        -- Catch any errors that occur during the procedure call
        DBMS_OUTPUT.PUT_LINE('Error calling delete_allowance: ' || SQLERRM);
END;
/
```

Implicit Cursor:

1. SQL%NOTFOUND,
2. SQL%ROWCOUNT

PL/SQL Anonymous Block with Row Count

```
-- Enable output
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    v_employee_id employees.employee_id%TYPE;  -- Employee ID
    v_first_name employees.first_name%TYPE;
    v_last_name employees.last_name%TYPE;

BEGIN
    -- Attempt to select an employee from department 30
    SELECT employee_id, first_name, last_name
    INTO v_employee_id, v_first_name, v_last_name
    FROM employees
    WHERE department_id = 30 AND ROWNUM = 1;

    -- Check if no employee was found
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found in department 30.');
        RETURN;  -- Exit the block if no employee is found
    END IF;

    -- Attempt to delete the selected employee
    DELETE FROM employees WHERE employee_id = v_employee_id;

    -- Output the number of rows deleted
    DBMS_OUTPUT.PUT_LINE('Rows deleted: ' || SQL%ROWCOUNT);

    -- Check if the delete operation didn't find or affect any rows
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee with ID ' || v_employee_id || ' was found for deletion.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Employee ID ' || v_employee_id || ' deleted successfully.');
    END IF;

EXCEPTION
    -- Handle unexpected errors
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

```
-- Calling the procedure for department 30
EXEC delete_employee_from_department(30);
```

Question - Implicit Cursor for Retrieving Employee Allowance Details

Task:
Write a PL/SQL block that retrieves allowance details for an employee in **Department 30**, identified by their **ROLE_ID** in the EMPLOYEES table. The block should:

1. Use an implicit cursor (SELECT INTO statement) to fetch:
- ALLOWANCE_ID
- ALLOWANCE_AMOUNT
- EFFECTIVE_DATE

from the **ALLOWANCES** table.

2. Handle possible outcomes as follows:
- If **one record is found**, display the allowance details and print:
  "Allowance found for employee in department 30."
- If **no record is found**, use SQL%NOTFOUND or exception handling to print:
  "No allowance found for employee in department 30."
- If **multiple records are returned**, handle the TOO_MANY_ROWS exception and print:
  "Too many allowances found for employee in department 30."
- For any other errors, display the error message using SQLERRM.

3. Use DBMS_OUTPUT.PUT_LINE for all output messages.

Solution – Implicit Cursor
1. SQL%NOTFOUND
2. SQL%ROWCOUNT

Using a Cursor to Fetch Employees by Department

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    -- Variables to store allowance details
    v_allowance_id      ALLOWANCES.ALLOWANCE_ID%TYPE;
    v_allowance_amount  ALLOWANCES.ALLOWANCE_AMOUNT%TYPE;
    v_effective_date    ALLOWANCES.EFFECTIVE_DATE%TYPE;
    v_employee_role_id  EMPLOYEES.ROLE_ID%TYPE;

BEGIN
    -- Retrieve one employee's ROLE_ID from Department 2
    SELECT role_id
    INTO v_employee_role_id
    FROM employees
    WHERE department_id = 2
    AND ROWNUM = 1; --Return the first row that meets the condition (in this case, where department_id
= 2).

    -- Retrieve allowance details for the employee (implicit cursor)
    SELECT allowance_id, allowance_amount, effective_date
    INTO v_allowance_id, v_allowance_amount, v_effective_date
    FROM allowances
    WHERE role_id = v_employee_role_id;

    -- Check if data was found using SQL%FOUND
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Allowance found for employee in department 2.');
        DBMS_OUTPUT.PUT_LINE('Allowance ID: ' || v_allowance_id);
        DBMS_OUTPUT.PUT_LINE('Allowance Amount: ' || v_allowance_amount);
        DBMS_OUTPUT.PUT_LINE('Effective Date: ' || TO_CHAR(v_effective_date, 'YYYY-MM-DD'));
        DBMS_OUTPUT.PUT_LINE('Rows affected: ' || SQL%ROWCOUNT);
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No allowance found for employee in department 30.');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Too many allowances found for employee in department 30.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

```
Allowance found for employee in department
30.
Allowance ID: 2
Allowance Amount: 5000
Effective Date: 2024-09-29
Rows affected: 1


PL/SQL procedure successfully completed.
```

PL/SQL Function with %ROWCOUNT and %NOTFOUND (1/2)

```
CREATE OR REPLACE PROCEDURE get_salary_statistics(
    p_department_id IN NUMBER,
    p_start_date IN DATE,
    p_end_date IN DATE,
    p_salary_threshold IN NUMBER
)
IS
    v_total_salary NUMBER;
    v_average_salary NUMBER;
    v_employee_count NUMBER;
    v_salary_message VARCHAR2(200);
BEGIN
    -- Calculate the total salary of employees in the department hired between p_start_date and
p_end_date
    SELECT NVL(SUM(salary), 0)
    INTO v_total_salary
    FROM employees
    WHERE department_id = p_department_id
    AND hire_date BETWEEN p_start_date AND p_end_date;

    -- Check if no rows were returned for the total salary calculation
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in department ' || p_department_id || ' for the
given date range.');
        RETURN;  -- Exit if no employees are found
    END IF;

    -- Calculate the average salary of employees in the department hired between p_start_date and
p_end_date
    SELECT NVL(AVG(salary), 0)
    INTO v_average_salary
    FROM employees
    WHERE department_id = p_department_id
    AND hire_date BETWEEN p_start_date AND p_end_date;
 -- Check if no rows were returned for the average salary calculation
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in department ' || p_department_id || ' for the
given date range.');
        RETURN;  -- Exit if no employees are found
    END IF;
```

PL/SQL Function with %ROWCOUNT and %NOTFOUND (2/2)

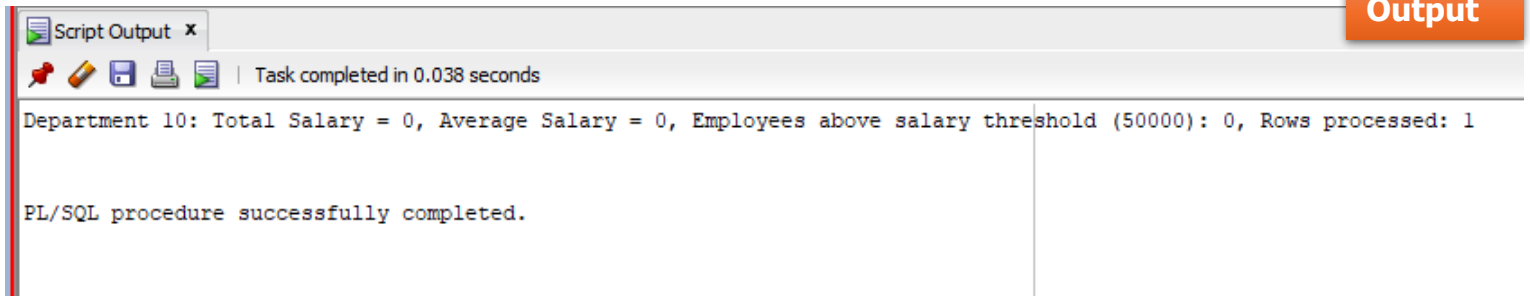
Store Procedure

```
-- Count employees who earn above the salary threshold
   SELECT COUNT(*)
   INTO v_employee_count
   FROM employees
   WHERE department_id = p_department_id
   AND hire_date BETWEEN p_start_date AND p_end_date
   AND salary > p_salary_threshold;

   -- Prepare the result message
   v_salary_message := 'Department ' || p_department_id || ': ' ||
                       'Total Salary = ' || v_total_salary || ', ' ||
                       'Average Salary = ' || v_average_salary || ', ' ||
                       'Employees above salary threshold (' || p_salary_threshold || '): ' || v_employee_count ||
                       ', Rows processed: ' || SQL%ROWCOUNT;

   -- Output the message
   DBMS_OUTPUT.PUT_LINE(v_salary_message);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employees found for the given criteria in department ' ||
p_department_id);
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Call the procedure

```
SET SERVEROUTPUT ON;

EXEC get_salary_statistics(10, TO_DATE('2024-01-01', 'YYYY-MM-DD'), TO_DATE('2024-12-31', 'YYYY-MM-DD'), 50000);
```

Output



Implicit Cursor SQL%ROWCOUNT

```
-- Enable output
SET SERVEROUTPUT ON SIZE 100000;

CREATE OR REPLACE PROCEDURE check_attendance(p_employee_id NUMBER, p_date DATE) AS
   v_attendance_status ATTENDANCE.STATUS%TYPE;  -- Variable to hold the attendance status
BEGIN
   -- Implicit cursor to select the attendance status of an employee on a specific date
   SELECT STATUS
   INTO v_attendance_status
   FROM ATTENDANCE
   WHERE EMPLOYEE_ID = p_employee_id AND ATTENDANCE_DATE = p_date;

   -- Check the number of rows returned by the implicit cursor
   IF SQL%ROWCOUNT = 1 THEN
      -- Output the attendance status if a record is found
      DBMS_OUTPUT.PUT_LINE('Attendance status for Employee ID ' || p_employee_id ||
                           ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || ': ' || v_attendance_status);
   ELSE
      DBMS_OUTPUT.PUT_LINE('No attendance record found for Employee ID ' || p_employee_id ||
                           ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || '.');
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      -- This exception will be triggered if no record is found for the specified employee and date
      DBMS_OUTPUT.PUT_LINE('No attendance record found for Employee ID ' || p_employee_id ||
                           ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || '.');
   WHEN OTHERS THEN
      -- Handle any other exceptions
      DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

```
--Call the check_attendance procedure
BEGIN
   check_attendance(p_employee_id => 101, p_date => TO_DATE('2024-11-01', 'YYYY-MM-DD'));
END;
/
```

PL/SQL Table Types (Associative Arrays)
A **PL/SQL table** (also known as an **associative array**) is a collection that can hold a set of values indexed by a number or string.
Example of using a PL/SQL table:

User defined Types or Type Variables

```
DECLARE
   TYPE employee_ids_table IS TABLE OF EMPLOYEES.EMPLOYEE_ID%TYPE INDEX BY BINARY_INTEGER;
   v_employee_ids employee_ids_table;  -- PL/SQL table variable
BEGIN
   -- Fetch employee IDs using BULK COLLECT
   SELECT employee_id
   BULK COLLECT INTO v_employee_ids
   FROM employees
   WHERE department_id = 2
   ORDER BY employee_id
   FETCH FIRST 2 ROWS ONLY;  -- Ensures only 2 rows are fetched (For Oracle 12c+)

   -- Check if the SQL query returned any rows
   IF SQL%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Query executed successfully. Number of employees fetched: ' || SQL%ROWCOUNT);

      -- Display the fetched employee IDs
      FOR i IN 1..v_employee_ids.COUNT LOOP
         DBMS_OUTPUT.PUT_LINE('Employee ' || i || ' ID: ' || v_employee_ids(i));
      END LOOP;
   ELSIF SQL%NOTFOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employees found in department 10.');
   END IF;

   -- Checking if the implicit SQL cursor is open (which is always FALSE for implicit cursors)
   IF SQL%ISOPEN THEN
      DBMS_OUTPUT.PUT_LINE('The implicit cursor is still open (unexpected behavior).');
   ELSE
      DBMS_OUTPUT.PUT_LINE('The implicit cursor is closed as expected.');
   END IF;

EXCEPTION
   -- Handle case where no data is found
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employees found in department 10.');

   -- Handle case where too many rows are fetched (not needed for BULK COLLECT but useful for single-row queries)
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Error: Query returned more than one row where only one was expected.');

   -- Handle any other unexpected errors
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

Output

```
Query executed successfully. Number of employees fetched: 2
Employee 1 ID: 1
Employee 2 ID: 4
The implicit cursor is closed as expected.
```

Optimizing PL/SQL Performance: BULK COLLECT vs FOR LOOP vs FORALL – When and Why to Use Each?

Why Use BULK COLLECT vs FOR LOOP?

1. BULK COLLECT (Your Current Approach)

- **Efficient for large datasets**: Fetches all rows at once and stores them in memory.
- **Minimizes context switches**: Reduces the number of times control moves between SQL and PL/SQL engines.
- **Faster execution**: Since all rows are fetched in one go, it's much faster compared to row-by-row processing.

2. FOR LOOP with CURSOR (Row-by-Row Processing)

- **Processes one row at a time**: Fetches data row by row inside the loop.
- **Higher context switching overhead**: Each iteration requires a back-and-forth between PL/SQL and SQL engines.
- **Better for small datasets**: Useful when memory is a concern since it does not load all rows at once.

Which One to Use?

- **Use BULK COLLECT** when dealing with large datasets to improve performance.
- **Use FOR LOOP with CURSOR** if you need to process rows one at a time or when working with limited memory.

BULK COLLECT vs FORALL in PL/SQL

Both BULK COLLECT and FORALL are used for performance optimization in PL/SQL, but they serve different purposes:

Feature	BULK COLLECT	FORALL
Purpose	Fetches multiple rows into a collection	Performs bulk DML operations (INSERT, UPDATE, DELETE)
Operation Type	SELECT (retrieval)	INSERT, UPDATE, DELETE (modifications)
Performance	Reduces context switching for queries	Reduces context switching for DML operations
Use Case	Reading large data sets efficiently	Executing multiple DML statements efficiently

✓ **BULK COLLECT** is used for efficiently retrieving large datasets.
✓ **FORALL** is used for efficiently modifying large datasets.
✓ **BULK_ROWCOUNT** helps track the number of rows affected in bulk operations.

BULK COLLECT

- **BULK COLLECT** is used to fetch multiple rows of data into a collection (like a nested table, VARRAY, or associative array) in one go.
- This eliminates the need for fetching rows one by one, which reduces the time spent in context switching between PL/SQL and SQL engines.

Efficient Employee Data Retrieval with Bulk Collect in PL/SQL

BULK COLLECTION

```
DECLARE
  -- Define a collection type based on the employees table's row structure
  TYPE emp_table IS TABLE OF employees%ROWTYPE;

  -- Declare a variable to hold the collection of employee data
  emp_data emp_table;
BEGIN
  -- Fetch all employees from the employees table into the collection using BULK COLLECT
  SELECT * BULK COLLECT INTO emp_data FROM employees;

  -- Check if the implicit cursor is open and if data was found
  IF SQL%ISOPEN THEN
    -- If data was found, print the number of rows fetched
    IF SQL%FOUND THEN
      DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employees found.');

      -- Loop through the collection and print details of each employee
      FOR i IN emp_data.FIRST .. emp_data.LAST LOOP
        DBMS_OUTPUT.PUT_LINE('Row ' || i || ': ' || emp_data(i).first_name || ' earns ' || emp_data(i).salary);
      END LOOP;

    -- If no data was found, print a message indicating no employees were found
    ELSIF SQL%NOTFOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employees found.');
    END IF;
  ELSE
    -- If the cursor is not open, print a message indicating the cursor is not open
    DBMS_OUTPUT.PUT_LINE('Cursor is not open.');
  END IF;

EXCEPTION
  -- Handle the case where no data was found (for SELECT statements returning no rows)
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees exist in the table.');

  -- Handle any other exceptions that occur during the execution of the block
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Bulk Collect + FORALL in PL/SQL

BULK COLLECT + FORALL for Bulk UPDATE

```
DECLARE
  -- Define a collection type based on the employees table's row structure
  TYPE emp_table IS TABLE OF employees%ROWTYPE;

  -- Declare a variable to hold the collection of employee data
  emp_data emp_table;

  -- Declare an array to hold update counts for bulk processing
  TYPE num_table IS TABLE OF NUMBER;
  update_counts num_table;

BEGIN
  -- Fetch all employees from the employees table into the collection using BULK COLLECT
  SELECT * BULK COLLECT INTO emp_data FROM employees;

  -- Check if rows were fetched using implicit cursor attributes
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employees found.');

    -- Loop through the collection and print details of each employee
    FOR i IN emp_data.FIRST .. emp_data.LAST LOOP
      DBMS_OUTPUT.PUT_LINE('Row ' || i || ': ' || emp_data(i).first_name || ' earns ' || emp_data(i).salary);
    END LOOP;
  ELSIF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees found.');
  END IF;

  -- Example of using FORALL to perform a bulk update (assuming an increment in salary)
  FORALL i IN emp_data.FIRST .. emp_data.LAST
    UPDATE employees
    SET salary = salary * 1.10  -- Increase salary by 10%
    WHERE employee_id = emp_data(i).employee_id
    RETURNING employee_id BULK COLLECT INTO update_counts;

  -- Check how many rows were actually updated using SQL%BULK_ROWCOUNT
  FOR i IN 1 .. update_counts.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Updated Employee ID: ' || update_counts(i) || ' - Rows Affected: ' || SQL%BULK_ROWCOUNT(i));
  END LOOP;

  -- Display the total count of updates
  DBMS_OUTPUT.PUT_LINE('Total updates performed: ' || update_counts.COUNT);

EXCEPTION
  -- Handle the case where no data was found (for SELECT statements returning no rows)
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees exist in the table.');

  -- Handle any other exceptions that occur during execution
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Explanation:

- BULK COLLECT INTO emp_ids: Collects **all employee IDs** from **department 20**.
- FORALL i IN 1 .. emp_ids.COUNT: **Updates all salaries in one operation**.
- **Efficient compared to row-by-row updates**.

Implicit Cursor SQL%BULK_ROWCOUNT

BULK_ROWCOUNT

```
DECLARE
    -- Define a nested table type to store a list of department IDs
    TYPE DeptList IS TABLE OF NUMBER;

    -- Initialize a nested table with sample Department IDs
    depts DeptList := DeptList(10, 20, 30);  -- Sample Department IDs
BEGIN
    -- Use FORALL to perform a bulk delete operation for each department ID in the depts table
    FORALL i IN depts.FIRST .. depts.LAST
        DELETE FROM EMPLOYEES WHERE DEPARTMENT_ID = depts(i);
    -- Loop through each department ID to display the number of rows deleted for each
department
    FOR i IN depts.FIRST .. depts.LAST LOOP
        DBMS_OUTPUT.PUT_LINE('Deletion in Department #' || depts(i) || ': ' ||
SQL%BULK_ROWCOUNT(i) || ' employees deleted.');
    END LOOP;
    -- Display the total number of employees deleted across all departments
    DBMS_OUTPUT.PUT_LINE('Total employees deleted: ' || SQL%ROWCOUNT);
END;
/
```

```
-- Returned by %ROWCOUNT
Total employees deleted: 10
```

```
-- Returned by %BULK_ROWCOUNT
Deletion in Department #10: 5 employees deleted.
Deletion in Department #20: 3 employees deleted.
Deletion in Department #30: 2 employees deleted.
```

SQL*Plus

FORALL

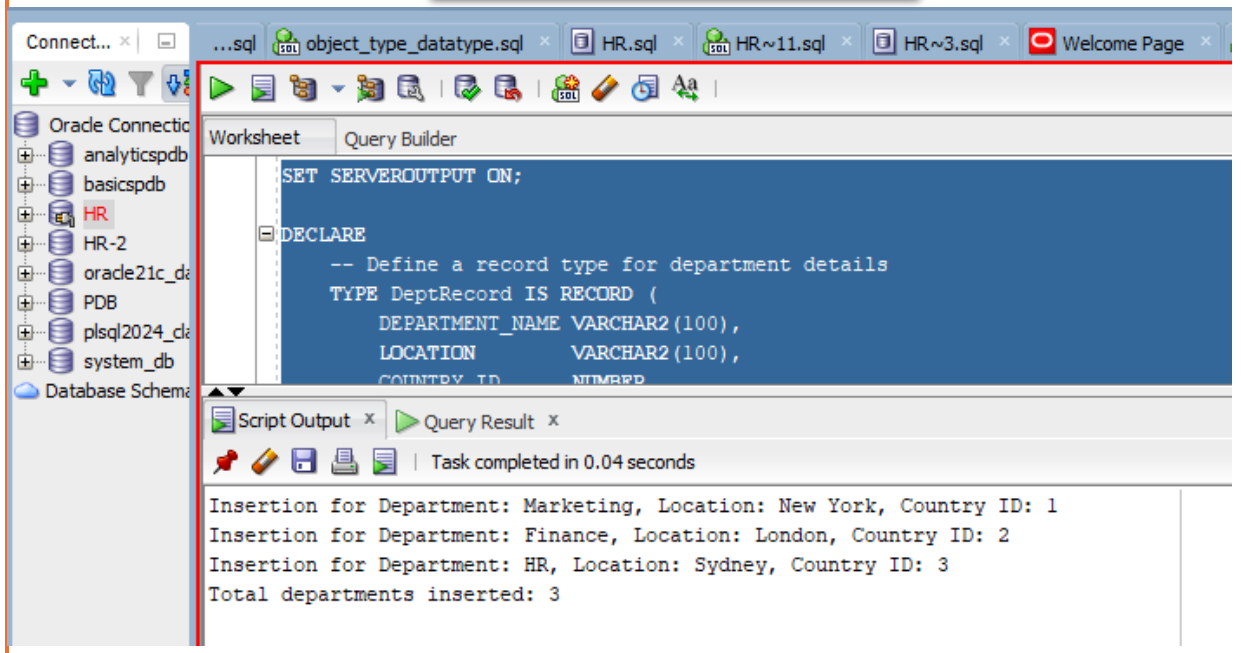
```
DECLARE
    -- Define a nested table type to store a list of department IDs
    TYPE DeptList IS TABLE OF NUMBER;

    -- Initialize a nested table with sample Department IDs
    depts DeptList := DeptList(10, 20, 30);  -- Sample Department IDs
BEGIN
    -- Loop through each department ID to perform deletion and display results
    FOR i IN depts.FIRST .. depts.LAST LOOP
        -- Perform delete operation for each department ID
        DELETE FROM EMPLOYEES WHERE DEPARTMENT_ID = depts(i);

        -- Display the number of rows deleted for each department
        DBMS_OUTPUT.PUT_LINE('Deletion in Department #' || depts(i) || ': ' ||
SQL%ROWCOUNT || ' employees deleted.');
    END LOOP;

    -- Display the total number of employees deleted across all departments
    DBMS_OUTPUT.PUT_LINE('Total employees deleted: ' || SQL%ROWCOUNT);
END;
/
```

Use Case: SQL%BULK_ROWCOUNT provides the **row count for each delete operation** per department, and **SQL%ROWCOUNT** gives the total number of rows deleted.

Implicit Cursor SQL%BULK_ROWCOUNT

```
DECLARE
    -- Define a record type for department details
    TYPE DeptRecord IS RECORD (
        DEPARTMENT_NAME VARCHAR2(100),
        LOCATION        VARCHAR2(100),
        COUNTRY_ID      NUMBER
    );
    -- Define a nested table type of the DeptRecord type
    TYPE DeptList IS TABLE OF DeptRecord;

    -- Initialize the nested table with department details
    depts DeptList := DeptList(
        DeptRecord('Marketing', 'New York', 1),
        DeptRecord('Finance', 'London', 2),
        DeptRecord('HR', 'Sydney', 3)
    );
BEGIN
    -- Perform bulk insert on the DEPARTMENTS table for each department in the depts table
    FORALL i IN depts.FIRST .. depts.LAST
        INSERT INTO DEPARTMENTS (DEPARTMENT_NAME, LOCATION, COUNTRY_ID)
        VALUES (depts(i).DEPARTMENT_NAME, depts(i).LOCATION, depts(i).COUNTRY_ID);

    -- Loop through each department to output the insertion status for each record
    FOR i IN depts.FIRST .. depts.LAST LOOP
        DBMS_OUTPUT.PUT_LINE('Insertion for Department: ' || depts(i).DEPARTMENT_NAME ||
                             ', Location: ' || depts(i).LOCATION ||
                             ', Country ID: ' || depts(i).COUNTRY_ID);
    END LOOP;

    -- Display the total number of departments inserted
    DBMS_OUTPUT.PUT_LINE('Total departments inserted: ' || SQL%ROWCOUNT);
END;
/
```

Output



Implicit Cursor SQL%BULK_EXCEPTIONS

```
DECLARE
    -- Define a collection type to hold a list of employee IDs as numbers
    TYPE employee_ids_t IS TABLE OF NUMBER;
    -- Initialize the collection with sample employee IDs
    employee_ids employee_ids_t := employee_ids_t(101, 102, 103, 104, 105);

    -- Variable to store the count of errors that occurred during the bulk operation
    errors NUMBER;
    -- Variable to use in loops for error handling
    i NUMBER;
BEGIN
    -- Perform bulk INSERT operation on the 'employees' table
    -- Using 'SAVE EXCEPTIONS' to capture individual row errors and continue with other rows
    FORALL j IN employee_ids.FIRST .. employee_ids.LAST SAVE EXCEPTIONS
        INSERT INTO employees (employee_id, salary)
        VALUES (employee_ids(j), 5000);  -- Assign a salary of 5000 for each employee ID
-- Exception handling block to catch and process any errors
EXCEPTION
    WHEN OTHERS THEN
        -- Get the count of exceptions that occurred during the FORALL operation
        errors := SQL%BULK_EXCEPTIONS.COUNT;

        -- Output the number of errors encountered
        DBMS_OUTPUT.PUT_LINE('Number of errors: ' || errors);

        -- Loop through each exception recorded in SQL%BULK_EXCEPTIONS
        FOR i IN 1 .. errors LOOP
            -- Output the index of the employee ID that caused the error and the error code
            DBMS_OUTPUT.PUT_LINE('Error at index ' ||
                                 SQL%BULK_EXCEPTIONS(i).ERROR_INDEX ||
                                 ': Error code ' ||
                                 SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
        END LOOP;
END;
```

Explanation of Key Sections

- **DECLARE Section**: This section defines the employee_ids_t type, initializes a collection employee_ids, and declares variables for error handling.
- **FORALL Statement**: The FORALL statement attempts to insert each employee ID from the collection into the employees table with a salary of 5000. The SAVE EXCEPTIONS clause is used to allow the operation to continue even if errors occur on individual rows.
- **Exception Handling**: The EXCEPTION block captures errors that arise from the FORALL operation. The SQL%BULK_EXCEPTIONS cursor attribute provides details about each error, including the error index (the row that caused the error) and the error code.
- **Output Error Information**: The loop within the EXCEPTION block outputs each error's index and code, allowing the developer to diagnose which specific rows failed and why.

This code is useful in scenarios where you want bulk processing to continue even if some records cause errors, and you need detailed error information for each failed row.

What Happens Internally:

1.The FORALL statement tries to insert rows:
- INSERT INTO employees (101, 5000) → **Error (primary key violation)**.
- INSERT INTO employees (102, 5000) → **Success**.
- INSERT INTO employees (103, 5000) → **Error (primary key violation)**.
- INSERT INTO employees (104, 5000) → **Success**.
- INSERT INTO employees (105, 5000) → **Success**.

2.Errors are saved, and successful inserts are committed.

3.The exception-handling block processes and outputs the errors, iterating over SQL%BULK_EXCEPTIONS.

```
Number of errors: 2
Error at index 1: Error code 1
Error at index 3: Error code 1
```

Implicit Cursor SQL%BULK_EXCEPTIONS

- **Explanation**: This attribute stores errors encountered during a **FORALL** operation when using the **SAVE EXCEPTIO**NS clause.
- **Example**:This example will handle errors gracefully, allowing successful records to be inserted and providing details on any that failed.

Explanation of Code:

Record and Table Types: EmpRecord holds individual employee details, and EmpList stores multiple records for bulk processing.

FORALL with SAVE EXCEPTIONS: Allows bulk insertion; any error during insertion is saved, and processing continues with remaining records.

Exception Handling: SQL%BULK_EXCEPTIONS stores details of each error, looping through to display each error's index and code.

Total Count: SQL%ROWCOUNT outputs the count of successfully inserted rows.

Notes: SQL%BULK_EXCEPTIONS.COUNT provides the total error count, while ERROR_INDEX identifies each failed record for debugging.

Implicit Cursor SQL%BULK_EXCEPTIONS

```
DECLARE
    -- Define a record type for employee details
    TYPE EmpRecord IS RECORD (
        FIRST_NAME   VARCHAR2(50),
        LAST_NAME    VARCHAR2(50),
        EMAIL        VARCHAR2(100),
        PHONE_NUMBER VARCHAR2(15),
        HIRE_DATE    DATE,
        DEPARTMENT_ID NUMBER,
        ROLE_ID      NUMBER,
        SALARY       NUMBER,
        BONUS        NUMBER
    );

    -- Define a nested table type of the EmpRecord type
    TYPE EmpList IS TABLE OF EmpRecord;

    -- Initialize the nested table with employee details
    employees EmpList := EmpList(
        EmpRecord('John', 'Doe', 'johndoe@example.com', '123-456-7890', SYSDATE, 10, 1, 50000, 5000),
        EmpRecord('Jane', 'Smith', 'janesmith@example.com', '098-765-4321', SYSDATE, 20, 2, 60000, 6000),
        EmpRecord('Robert', 'Brown', 'robertbrown@example.com', '555-555-5555', SYSDATE, 30, 3, 70000, 7000)
        -- Add more records as needed
    );

    -- Variable to count errors
    errors EXCEPTION;
    PRAGMA EXCEPTION_INIT(errors, -24381); -- Error code for bulk operation exceptions

BEGIN
    -- Perform bulk insert on the EMPLOYEES table for each employee in the employees table
    FORALL i IN employees.FIRST .. employees.LAST SAVE EXCEPTIONS
        INSERT INTO EMPLOYEES (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, DEPARTMENT_ID, ROLE_ID, SALARY, BONUS)
        VALUES (employees(i).FIRST_NAME, employees(i).LAST_NAME, employees(i).EMAIL, employees(i).PHONE_NUMBER,
                employees(i).HIRE_DATE, employees(i).DEPARTMENT_ID, employees(i).ROLE_ID, employees(i).SALARY, employees(i).BONUS);

    DBMS_OUTPUT.PUT_LINE('Total employees inserted: ' || SQL%ROWCOUNT);

EXCEPTION
    WHEN errors THEN
        -- Loop through each error in the SAVE EXCEPTIONS
        FOR i IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('Error at record ' || SQL%BULK_EXCEPTIONS(i).ERROR_INDEX ||
                                 ': Error Code - ' || SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
        END LOOP;
END;
/
```

Explicit Cursor

An explicit cursor in PL/SQL is a named cursor created by **the programmer to manage and control a query's context independently**. Unlike implicit cursors, which are automatically created by PL/SQL for single SQL statements, explicit cursors allow more control over multi-row queries and give access to each row's values in a controlled manner.

How Explicit Cursors Work

Explicit cursors are defined in three main steps:

Declare the cursor to define the SQL query.

Open the cursor to execute the query and initialize the result set.

Fetch rows one by one or in bulk from the cursor's result set.

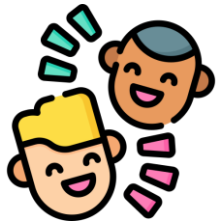
Close the cursor when done to release resources.

Key Benefits of Using Explicit Cursors

Row-by-Row Processing: Allows for processing rows one at a time in a controlled loop.

Flexible Error Handling: You can control what happens if no rows are returned or other exceptions occur.

Reusability: Cursors can be opened and fetched multiple times if needed (after closing them first).



Example of Explicit Cursor

```
1.  DECLARE
2.  CURSOR guru99_det IS SELECT emp_name FROM emp;
3.  lv_emp_name emp.emp_name%type;

4.  BEGIN
5.  OPEN guru99_det;

6.  LOOP
7.  FETCH guru99_det INTO lv_emp_name;
8.  IF guru99_det%NOTFOUND
9.  THEN
10. EXIT;
11. END IF;
12. Dbms_output.put_line ('Employee Fetched: ' || lv_emp_name);
13. END LOOP;
14. Dbms_output.put_line ('Total rows fetched is '||guru99_det%ROWCOUNT);
15. CLOSE guru99_det;
16. END;
17. /
```

Cursor declaration

Opening cursor

Loop to Fetch Cursor Data

Closing cursor

Output:

```
Employee Fetched: BBB
Employee Fetched: XXX
Employee Fetched: YYY
Total rows fetched is 3
```

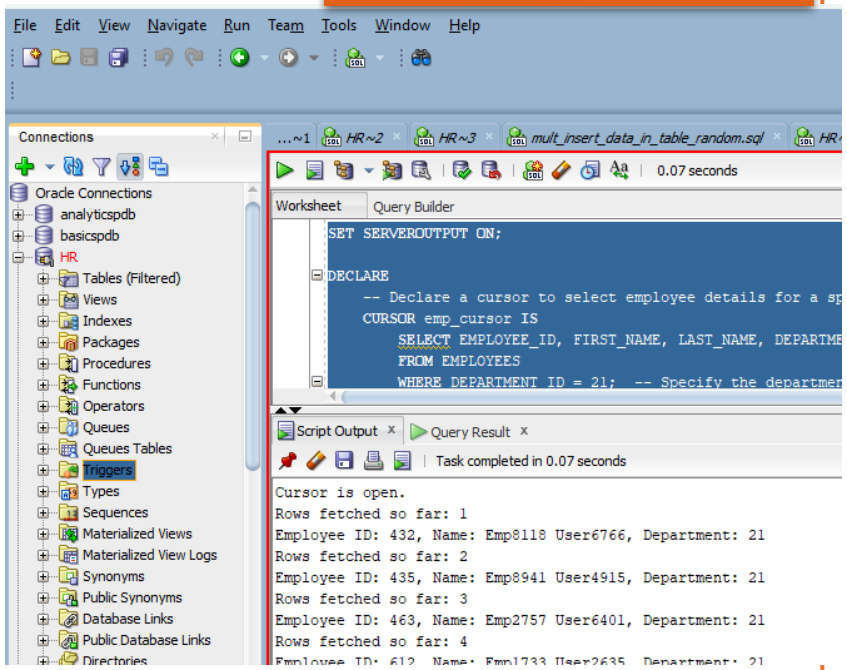
Source Image: https://www.guru99.com/pl-sql-cursor.html

SQL*Plus

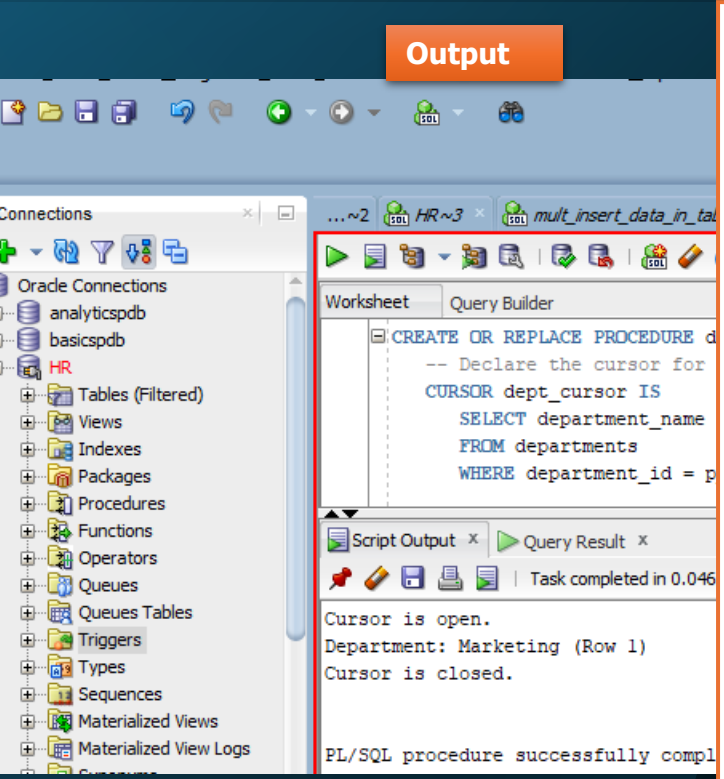
Explicit Cursor - Updated PL/SQL block with emp_cursor%ROWCOUNT:

```
SET SERVEROUTPUT ON;

DECLARE
    -- Declare a cursor to select employee details for a specific department
    CURSOR emp_cursor IS
        SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, DEPARTMENT_ID
        FROM EMPLOYEES
        WHERE DEPARTMENT_ID = 21; -- Specify the department

    -- Variables to hold the fetched data
    v_employee_id EMPLOYEES.EMPLOYEE_ID%TYPE;
    v_first_name  EMPLOYEES.FIRST_NAME%TYPE;
    v_last_name   EMPLOYEES.LAST_NAME%TYPE;
    v_department_id EMPLOYEES.DEPARTMENT_ID%TYPE;

BEGIN
    -- Open the cursor
    OPEN emp_cursor;

    -- Check if the cursor is open
    IF emp_cursor%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor is open.');
    END IF;
    -- Fetch each row and process the data
    LOOP
        -- Fetch data into variables
        FETCH emp_cursor INTO v_employee_id, v_first_name, v_last_name, v_department_id;

        -- Exit the loop when no more rows are returned
        EXIT WHEN emp_cursor%NOTFOUND;

        -- Display the number of rows fetched so far
        DBMS_OUTPUT.PUT_LINE('Rows fetched so far: ' || emp_cursor%ROWCOUNT);

        -- Display employee details
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id || ', Name: ' || v_first_name || ' ' || v_last_name || ', Department: ' ||
v_department_id);
    END LOOP;
    -- Close the cursor
    CLOSE emp_cursor;

    -- Check if the cursor is closed
    IF NOT emp_cursor%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor is closed.');
    END IF;

END;
/
```

Output



Output



Fetch and Display Department Names with Cursor and Exception Handling

```
CREATE OR REPLACE PROCEDURE display_departments(p_department_id IN NUMBER) AS
  -- Declare the cursor for retrieving department names based on department_id
  CURSOR dept_cursor IS
    SELECT department_name
    FROM departments
    WHERE department_id = p_department_id;

  -- Variable to hold each department name fetched by the cursor
  v_department_name departments.department_name%TYPE;

BEGIN
  -- Open the cursor
  OPEN dept_cursor;

  -- Check if the cursor is open
  IF dept_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is open.');
  END IF;

  -- Loop through each row fetched by the cursor
  LOOP
    FETCH dept_cursor INTO v_department_name;

    -- Exit the loop if no more rows are fetched
    EXIT WHEN dept_cursor%NOTFOUND;

    -- Check if a row is found and output the department name
    IF dept_cursor%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Department: ' || v_department_name || ' (Row ' ||
dept_cursor%ROWCOUNT || ')');
    END IF;
  END LOOP;

  -- Close the cursor after processing
  CLOSE dept_cursor;

  -- Check if the cursor is closed
  IF NOT dept_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is closed.');
  END IF;
```

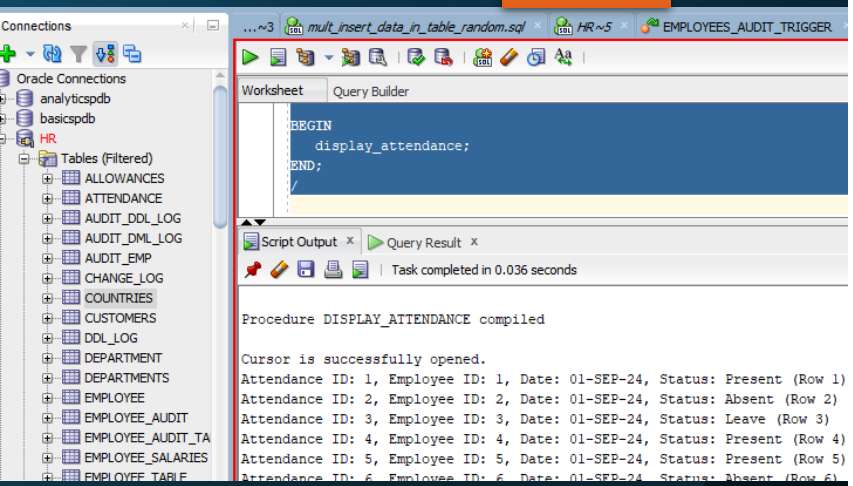
Call Procedure

```
BEGIN
  display_departments(26); -- Pass the department_id 26
END;
/
```

```
EXCEPTION
  -- Handle the case where no rows are found for the given department_id
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No departments found for department_id ' || p_department_id);

  -- Handle the case where there are too many rows returned
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Too many rows returned for department_id ' || p_department_id);

  -- Handle any other exceptions
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END display_departments;
/
```

Display_attendance procedure that includes the use of %ISOPEN, %FOUND, %NOTFOUND, and %ROWCOUNT

Output




```
CREATE OR REPLACE PROCEDURE display_attendance AS
  -- Declare the cursor for retrieving attendance records
  CURSOR attendance_cursor IS
    SELECT ATTENDANCE_ID, EMPLOYEE_ID, ATTENDANCE_DATE, STATUS
    FROM attendance;

  -- Variables to hold the attendance data fetched by the cursor
  v_attendance_id attendance.ATTENDANCE_ID%TYPE;
  v_employee_id   attendance.EMPLOYEE_ID%TYPE;
  v_attendance_date attendance.ATTENDANCE_DATE%TYPE;
  v_status        attendance.STATUS%TYPE;

BEGIN
  -- Open the cursor
  OPEN attendance_cursor;
  -- Check if the cursor is open
  IF attendance_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is successfully opened.');
  END IF;

  -- Loop through each row fetched by the cursor
  LOOP
    FETCH attendance_cursor INTO v_attendance_id, v_employee_id, v_attendance_date, v_status;
    -- Exit the loop if no more rows are fetched
    EXIT WHEN attendance_cursor%NOTFOUND;

    -- Check if a row was fetched and output the attendance record
    IF attendance_cursor%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Attendance ID: ' || v_attendance_id ||
                           ', Employee ID: ' || v_employee_id ||
                           ', Date: ' || v_attendance_date ||
                           ', Status: ' || v_status ||
                           ' (Row ' || attendance_cursor%ROWCOUNT || ')');
    END IF;
  END LOOP;
  -- Close the cursor after processing
  CLOSE attendance_cursor;

  -- Check if the cursor is closed
  IF NOT attendance_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is successfully closed.');
  END IF;
```

Call Procedure

```
BEGIN
  display_attendance;
END;
/
```

```
--Exception handling
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No attendance records found.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: '
|| SQLERRM);
END display_attendance;
/
```

Using Explicit Cursor to Retrieve Employee Data with ROWNUM & Exception Handling

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    -- Define the cursor to retrieve employees from department 1 (up to 2 employees)
    CURSOR emp_cursor IS
        SELECT employee_id, first_name, last_name
        FROM employees
        WHERE department_id = 1 AND ROWNUM <= 2;  -- Adjust the condition as needed

BEGIN
    -- Loop through each row fetched by the cursor
    FOR emp_record IN emp_cursor LOOP
        -- Output the employee details
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id ||
                             ', Name: ' || emp_record.first_name || ' ' || emp_record.last_name);
    END LOOP;

    -- Since the FOR loop automatically handles cursor operations, no need to check %FOUND here.

EXCEPTION
    -- Handle the case where no data is found
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found in department 1.');
    -- Handle any other exceptions
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Key Changes:

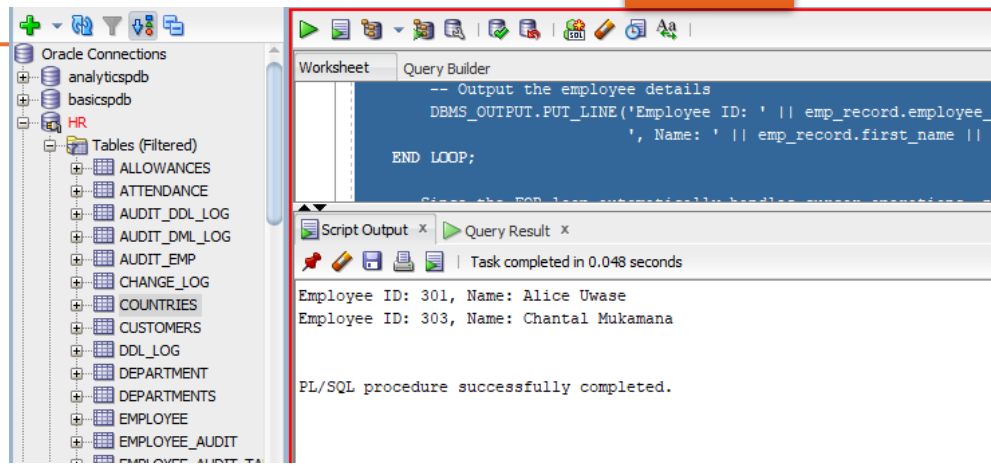
1. FOR Loop with Cursor:

- The FOR emp_record IN emp_cursor LOOP automatically opens the cursor and fetches the rows until the cursor is exhausted.
- There's no need to explicitly check %FOUND or %NOTFOUND as the FOR loop does that internally.

2. Error Handling:

- If no rows are returned, the NO_DATA_FOUND exception will not trigger because a FOR loop will not even run if there are no records. However, other exceptions (e.g., SQL errors) will be caught in the OTHERS exception block.

Output



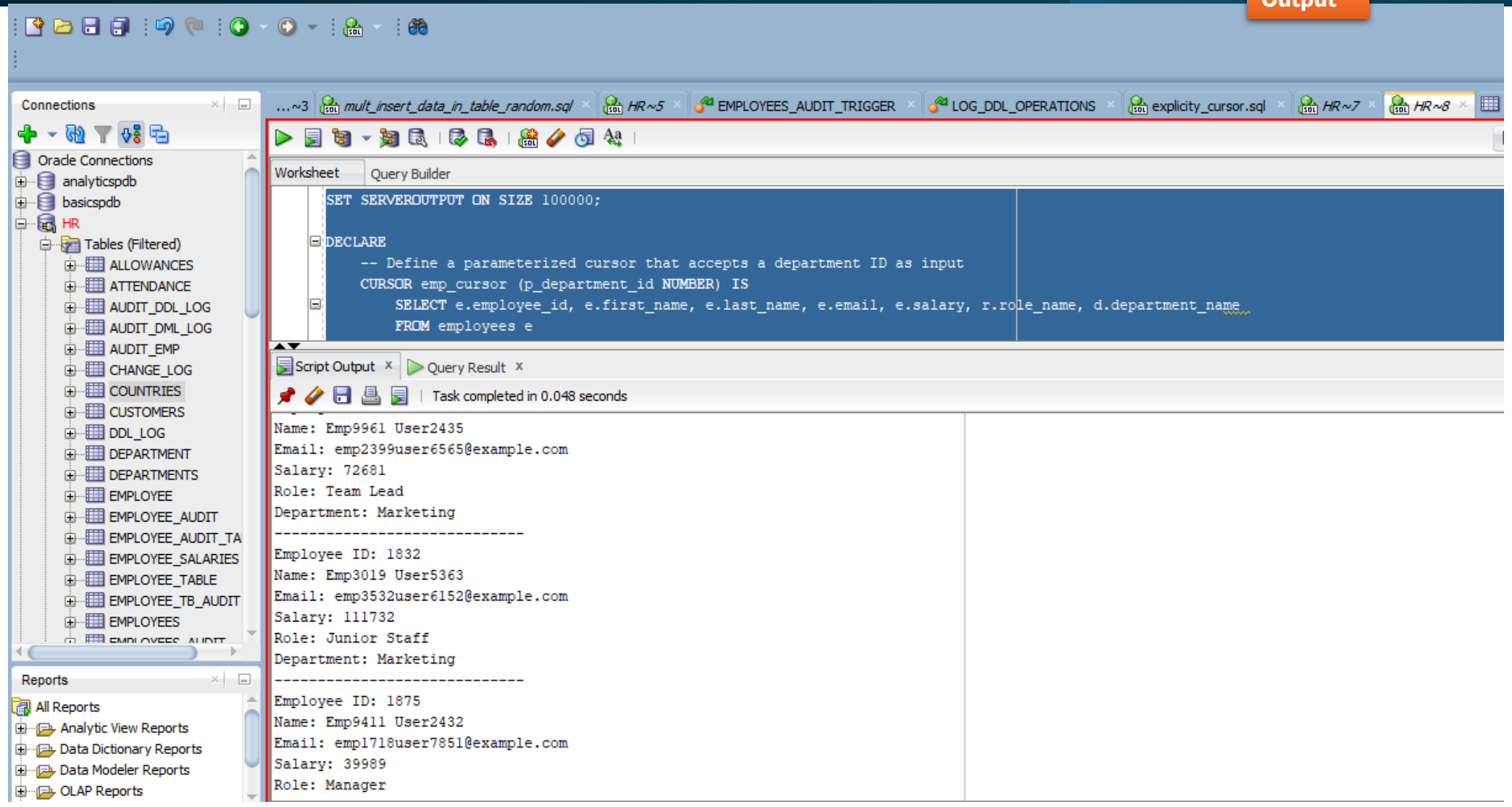
Using Parameterized Cursor for Employee Details by Department

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    -- Define a parameterized cursor that accepts a department ID as input
    CURSOR emp_cursor (p_department_id NUMBER) IS
        SELECT e.employee_id, e.first_name, e.last_name, e.email, e.salary, r.role_name,
d.department_name
        FROM employees e
        JOIN roles r ON e.role_id = r.role_id
        JOIN departments d ON e.department_id = d.department_id
        WHERE e.department_id = p_department_id;  -- Filter based on the given department ID

    -- Variables to store employee details
    v_employee_id employees.employee_id%TYPE;
    v_first_name employees.first_name%TYPE;
    v_last_name employees.last_name%TYPE;
    v_email employees.email%TYPE;
    v_salary employees.salary%TYPE;
    v_role_name roles.role_name%TYPE;
    v_department_name departments.department_name%TYPE;

BEGIN
    -- Open the cursor with a specific department ID, e.g., department 10
    OPEN emp_cursor(21);

    -- Check if the cursor is open
    IF emp_cursor%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor opened successfully.');
    END IF;

    LOOP
        -- Fetch each row into the variables
        FETCH emp_cursor INTO v_employee_id, v_first_name, v_last_name, v_email, v_salary,
v_role_name, v_department_name;
```

```
        -- Exit when no more rows are returned
        EXIT WHEN emp_cursor%NOTFOUND;

        -- Display employee and department details
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_first_name || ' ' || v_last_name);
        DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
        DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
        DBMS_OUTPUT.PUT_LINE('Role: ' || v_role_name);
        DBMS_OUTPUT.PUT_LINE('Department: ' || v_department_name);
        DBMS_OUTPUT.PUT_LINE('-----------------------------');
    END LOOP;

    -- Check if any rows were fetched using the %FOUND attribute
    IF emp_cursor%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Rows fetched successfully for department ' || 10);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No employees found in department ' || 10);
    END IF;

    -- Close the cursor after processing
    CLOSE emp_cursor;

    -- Check if the cursor is closed
    IF NOT emp_cursor%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor is successfully closed.');
    END IF;

EXCEPTION
    -- Handle any exceptions
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data found for department ' || 10);
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Using Parameterized Cursor for Employee Details by Department

Output



Explicit Cursor Handling with DBMS_SQL in Oracle

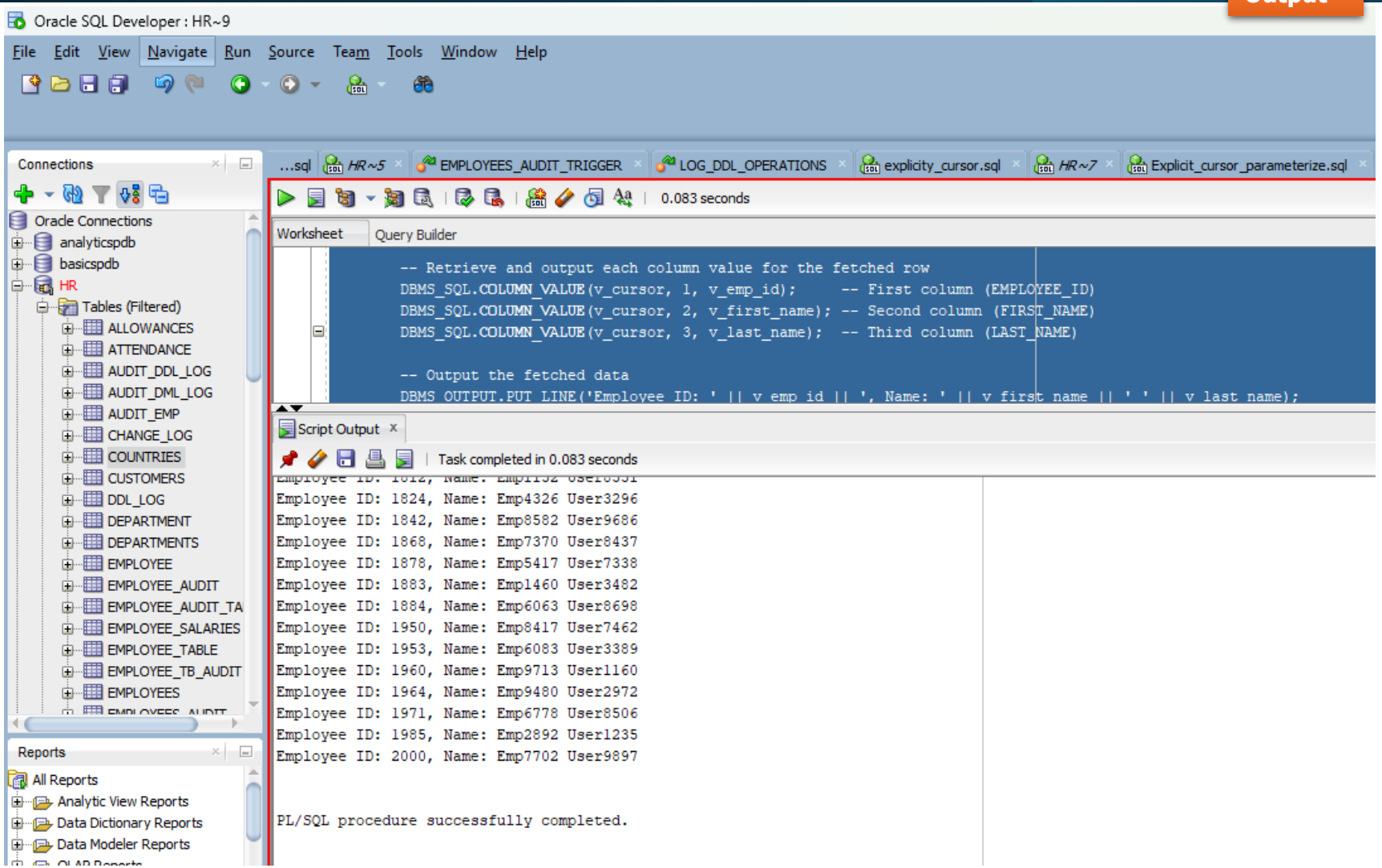
```
DECLARE
  v_cursor NUMBER;
  v_status NUMBER;
  v_employee_id NUMBER;
  v_first_name VARCHAR2(50);
  v_last_name VARCHAR2(50);
  v_dept_id NUMBER := 3;
BEGIN
  -- Open Cursor
  v_cursor := DBMS_SQL.OPEN_CURSOR;

  -- Parse SQL Query
  DBMS_SQL.PARSE(v_cursor, 'SELECT employee_id, first_name, last_name FROM employees WHERE department_id = :dept_id',
DBMS_SQL.NATIVE);

  -- Bind Variables
  DBMS_SQL.BIND_VARIABLE(v_cursor, ':dept_id', v_dept_id);

  -- Define Columns
  DBMS_SQL.DEFINE_COLUMN(v_cursor, 1, v_employee_id);
  DBMS_SQL.DEFINE_COLUMN(v_cursor, 2, v_first_name, 50);
  DBMS_SQL.DEFINE_COLUMN(v_cursor, 3, v_last_name, 50);

  -- Execute SQL
  v_status := DBMS_SQL.EXECUTE(v_cursor);

  -- Fetch Rows
  WHILE DBMS_SQL.FETCH_ROWS(v_cursor) > 0 LOOP
    DBMS_SQL.COLUMN_VALUE(v_cursor, 1, v_employee_id);
    DBMS_SQL.COLUMN_VALUE(v_cursor, 2, v_first_name);
    DBMS_SQL.COLUMN_VALUE(v_cursor, 3, v_last_name);

    -- Output Result
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id || ', Name: ' || v_first_name || ' ' || v_last_name);
  END LOOP;

  -- Close Cursor
  DBMS_SQL.CLOSE_CURSOR(v_cursor);
EXCEPTION
  WHEN OTHERS THEN
    IF DBMS_SQL.IS_OPEN(v_cursor) THEN
      DBMS_SQL.CLOSE_CURSOR(v_cursor);
    END IF;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Explicit Cursor - Dynamic SQL Cursor Management

```
SET SERVEROUTPUT ON;

DECLARE
    v_cursor     INT;  -- Cursor handle
    v_emp_id     EMPLOYEES.EMPLOYEE_ID%TYPE;
    v_first_name EMPLOYEES.FIRST_NAME%TYPE;
    v_last_name  EMPLOYEES.LAST_NAME%TYPE;
    v_status     INT;  -- To store the status of the cursor execution
    v_row_count  INT;  -- To store the row count after fetching

BEGIN
    -- Open a cursor for a dynamic SQL query
    v_cursor := DBMS_SQL.OPEN_CURSOR;

    -- Parse the dynamic SQL (query string)
    DBMS_SQL.PARSE(v_cursor, 'SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME FROM
EMPLOYEES WHERE DEPARTMENT_ID = :dept_id', DBMS_SQL.NATIVE);

    -- Bind the variable (department_id) to the placeholder in the query
    DBMS_SQL.BIND_VARIABLE(v_cursor, ':dept_id', 3);

    -- Define columns with specific data types to match the SELECT statement columns
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 1, v_emp_id);           -- Numeric type for
EMPLOYEE_ID
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 2, v_first_name, 50);   -- VARCHAR2 type with
length 50 for FIRST_NAME
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 3, v_last_name, 50);    -- VARCHAR2 type with
length 50 for LAST_NAME

    -- Execute the query
    v_status := DBMS_SQL.EXECUTE(v_cursor);
    DBMS_OUTPUT.PUT_LINE('Execution status: ' || v_status);

    -- Fetch and output each row's data until no rows remain
    LOOP
        v_row_count := DBMS_SQL.FETCH_ROWS(v_cursor);
        EXIT WHEN v_row_count = 0;

        -- Retrieve and output each column value for the fetched row
        DBMS_SQL.COLUMN_VALUE(v_cursor, 1, v_emp_id);     -- First column (EMPLOYEE_ID)
        DBMS_SQL.COLUMN_VALUE(v_cursor, 2, v_first_name); -- Second column (FIRST_NAME)
        DBMS_SQL.COLUMN_VALUE(v_cursor, 3, v_last_name);  -- Third column (LAST_NAME)

        -- Output the fetched data
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp_id || ', Name: ' || v_first_name || ' '
|| v_last_name);
    END LOOP;

    -- Close the cursor after use
    DBMS_SQL.CLOSE_CURSOR(v_cursor);

EXCEPTION
    WHEN OTHERS THEN
        -- Handle any exceptions and close the cursor if still open
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
        IF DBMS_SQL.IS_OPEN(v_cursor) THEN
            DBMS_SQL.CLOSE_CURSOR(v_cursor);
        END IF;
END;
/
```

Explicit Cursor - DBMS_SQL cursor Handle – Example II

Output



Using Cursors and Functions to Fetch and Process Employee Data

Function with cursor

```
-- Function to fetch employee data and return a SYS_REFCURSOR
CREATE OR REPLACE FUNCTION get_all_employee_data
RETURN SYS_REFCURSOR
IS
  -- Declare a cursor variable to hold the result set
  emp_cursor SYS_REFCURSOR;
BEGIN
  -- Open the cursor for a SELECT query to fetch specific columns from the 'employees' table
  OPEN emp_cursor FOR
    SELECT employee_id, first_name, bonus, salary FROM employees;

  -- Return the cursor so it can be used in the calling block
  RETURN emp_cursor;
END;
/
```

Call the function

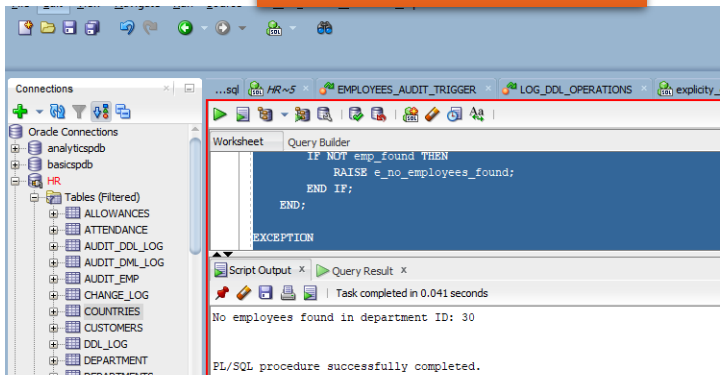
```
-- Anonymous PL/SQL block to call the function and process the employee data
DECLARE
  -- Declare a cursor variable to hold the result of the function call
  c_emp_data SYS_REFCURSOR;

  -- Declare variables to hold the values fetched from the cursor (using %TYPE for data type consistency)
  emp_id      employees.employee_id%TYPE;    -- Variable to store employee ID (data type from employees table)
  emp_name    employees.first_name%TYPE;     -- Variable to store employee name (data type from employees table)
  emp_bonus   employees.bonus%TYPE;          -- Variable to store employee bonus (data type from employees table)
  emp_salary employees.salary%TYPE;          -- Variable to store employee salary (data type from employees table)
BEGIN
  -- Call the function to get the cursor containing employee data
  c_emp_data := get_all_employee_data;

  -- Fetch each row from the cursor and process it
  LOOP
    -- Fetch the current row into the declared variables
    FETCH c_emp_data INTO emp_id, emp_name, emp_bonus, emp_salary;

    -- Exit the loop when no more rows are found
    EXIT WHEN c_emp_data%NOTFOUND;

    -- Process each record, here we output the data using DBMS_OUTPUT
    DBMS_OUTPUT.PUT_LINE('ID: ' || emp_id || ', Name: ' || emp_name ||
                         ', Bonus: ' || emp_bonus || ', Salary: ' || emp_salary);
  END LOOP;

  -- Close the cursor once all rows have been processed
  CLOSE c_emp_data;
END;
/
```

Using a Cursor to Fetch Employees by Department

Explicit Cursor FOR Loops I

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
   -- Cursor to fetch employee details from department_id = 30
   CURSOR c_emp_cursor IS
      SELECT employee_id, last_name, first_name, email, phone_number, hire_date
      FROM employees
      WHERE department_id = 30;

   -- Custom exception to handle the case where no employees are found
   e_no_employees_found EXCEPTION;

BEGIN
   -- Initialize a counter to track if the loop finds any records
   DECLARE
      emp_found BOOLEAN := FALSE;  -- Flag to check if any employee is found
   BEGIN
      -- Loop through each record returned by the cursor
      FOR emp_record IN c_emp_cursor LOOP
         -- Set the flag to TRUE if at least one record is found
         emp_found := TRUE;

         -- Output the employee details
         DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id ||
                              ', Name: ' || emp_record.first_name || ' ' || emp_record.last_name ||
                              ', Email: ' || emp_record.email ||
                              ', Phone: ' || emp_record.phone_number ||
                              ', Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-MM-DD'));
      END LOOP;

      -- Check if no employees were found and raise an exception
      IF NOT emp_found THEN
         RAISE e_no_employees_found;
      END IF;
   END;

EXCEPTION
   WHEN e_no_employees_found THEN
      DBMS_OUTPUT.PUT_LINE('No employees found in department ID: 30');
   WHEN OTHERS THEN
      -- Handle any unexpected errors
      DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Output



Question: Explicitly Cursor with PLSQL

Question: Write a PL/SQL function that calculates the annual salary of each employee in the department with Department_ID = 3. The function should return a result set containing the following information for each employee:

- Concatenated full name (first name and last name)
- Monthly salary
- Calculated annual salary

After creating the function, display the output by listing all employees in department ID 3 with their respective annual salaries.

Solution: Steps to Define the PL/SQL Function get_annual_salary

Define the Function: Create a PL/SQL function named get_annual_salary.

Declare Variables: Declare variables for employee information (first name, last name, monthly salary, annual salary).

Use an Explicit Cursor: Define an explicit cursor to fetch employees in Department_ID = 3, retrieving their monthly salary and names.

Open the Cursor: Open the cursor to start retrieving data.

Fetch Data: Loop through the cursor results to:

1.Concatenate first and last names for the full name.

2.Calculate the annual salary by multiplying the monthly salary by 12.

Store and Return Results: Collect the full name, monthly salary, and annual salary, then return this result set.

Display Results: Call the function to display all employees in department ID 3 along with their respective annual salaries.

Solution: Cursor - Create function

```
--Step 1: Create the Function with Comments

CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
  RETURN SYS_REFCURSOR
IS
  -- Declare a cursor variable to return the result set
  annual_salary_cursor SYS_REFCURSOR;
BEGIN
  -- Open a cursor for the result set containing employee's full name, monthly salary, and calculated annual
salary
  OPEN annual_salary_cursor FOR
    SELECT first_name || ' ' || last_name AS full_name,  -- Concatenate first name and last name
           salary AS monthly_salary,                     -- Select the monthly salary and alias it
           salary * 12 AS annual_salary                  -- Calculate annual salary by multiplying monthly salary by 12
    FROM employees                                       -- From the employees table
    WHERE department_id = p_department_id;               -- Filter results by the provided department ID

  -- Return the cursor containing the result set
  RETURN annual_salary_cursor;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
    RETURN NULL;
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    RETURN NULL;
END;
/
```

Solution: Cursor - Create function

Call the function

```
DECLARE
    v_cursor SYS_REFCURSOR;
    v_full_name VARCHAR2(200);
    v_monthly_salary NUMBER;
    v_annual_salary NUMBER;
BEGIN
    -- Call the function and store the returned cursor in v_cursor
    v_cursor := get_annual_salary(3);  -- Replace '3' with the desired department ID

    -- Loop through the cursor and fetch data
    LOOP
        FETCH v_cursor INTO v_full_name, v_monthly_salary, v_annual_salary;
        EXIT WHEN v_cursor%NOTFOUND;

        -- Display the fetched data
        DBMS_OUTPUT.PUT_LINE('Employee: ' || v_full_name ||
                             ', Monthly Salary: ' || v_monthly_salary ||
                             ', Annual Salary: ' || v_annual_salary);
    END LOOP;

    -- Close the cursor after fetching all records
    CLOSE v_cursor;
END;
/
```

Question: Explicit Cursor, Type Object, Explicit Exception

Question:
Write the PL/SQL code to implement the get_annual_salary function according to the above requirements. Also, write an anonymous block to call this function with a department ID, display the result, and handle any exceptions that may occur during the function execution.

Follow-up Questions:

1. Why is an explicit cursor used in this function instead of an implicit one?
2. What advantages does returning an emp_salary_table object type provide in terms of data handling and readability?
3. Describe how each exception (NO_DATA_FOUND, INVALID_CURSOR, OTHERS) is handled and explain why such detailed exception handling is necessary in PL/SQL functions.

Step 1: Define the Object and Table Types (if not already created)

```
-- Create an object type to represent individual employee salary details
CREATE OR REPLACE TYPE emp_salary_obj AS OBJECT (
   full_name       VARCHAR2(100),  -- Full name of the employee (first name and last name concatenated)
   monthly_salary  NUMBER,         -- Monthly salary of the employee
   annual_salary   NUMBER          -- Calculated annual salary (monthly salary multiplied by 12)
);
/

-- Create a table type to hold multiple emp_salary_obj objects
CREATE OR REPLACE TYPE emp_salary_table AS TABLE OF emp_salary_obj;
/
```

Solution I – Loop

```
CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
  RETURN emp_salary_table
IS
  -- Declare a variable of the table type to store results
  emp_salaries emp_salary_table := emp_salary_table();
  -- Declare an explicit cursor for fetching employee data
  CURSOR emp_cursor IS
    SELECT first_name || ' ' || last_name AS full_name,
           salary AS monthly_salary,
           salary * 12 AS annual_salary
    FROM employees
    WHERE department_id = p_department_id;

  -- Variables to store individual column values
  v_full_name       VARCHAR2(100);
  v_monthly_salary  NUMBER;
  v_annual_salary   NUMBER;
BEGIN
  -- Open the cursor
  OPEN emp_cursor;
  -- Fetch each row from the cursor and populate the emp_salaries table
  LOOP
    FETCH emp_cursor INTO v_full_name, v_monthly_salary, v_annual_salary;
    -- Exit when there are no more rows to fetch
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Add each fetched row as an object to the emp_salaries table
    emp_salaries.EXTEND;
    emp_salaries(emp_salaries.COUNT) := emp_salary_obj(v_full_name, v_monthly_salary, v_annual_salary);
  END LOOP;
  -- Close the cursor
  CLOSE emp_cursor;
  -- Return the populated emp_salaries table
  RETURN emp_salaries;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
    RETURN emp_salary_table();  -- Return an empty table if no data is found
  WHEN INVALID_CURSOR THEN
    DBMS_OUTPUT.PUT_LINE('Error: Attempted to operate on an unopened or closed cursor.');
    RETURN emp_salary_table();  -- Return an empty table if there's an invalid cursor operation
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    RETURN emp_salary_table();  -- Return an empty table for any other errors
END;
/
```

Solution II – Bulk Collection

```
-- Create the object type to hold employee salary details
CREATE OR REPLACE TYPE emp_salary_obj AS OBJECT (
    full_name        VARCHAR2(200),
    monthly_salary   NUMBER,
    annual_salary    NUMBER
);
/

-- Create the table type to hold a collection of employee salary objects
CREATE OR REPLACE TYPE emp_salary_table AS TABLE OF emp_salary_obj;
/
```

Additional Notes

- **Bulk Collection without Loop**: Using BULK COLLECT allows for the elimination of the explicit LOOP, making the function code shorter and more efficient.
- **Memory Considerations**: BULK COLLECT is suitable for moderate data volumes. For very large datasets, consider using LIMIT to fetch data in chunks to avoid memory issues.

Solution II – Bulk Collection

Solution II – Bulk Collection

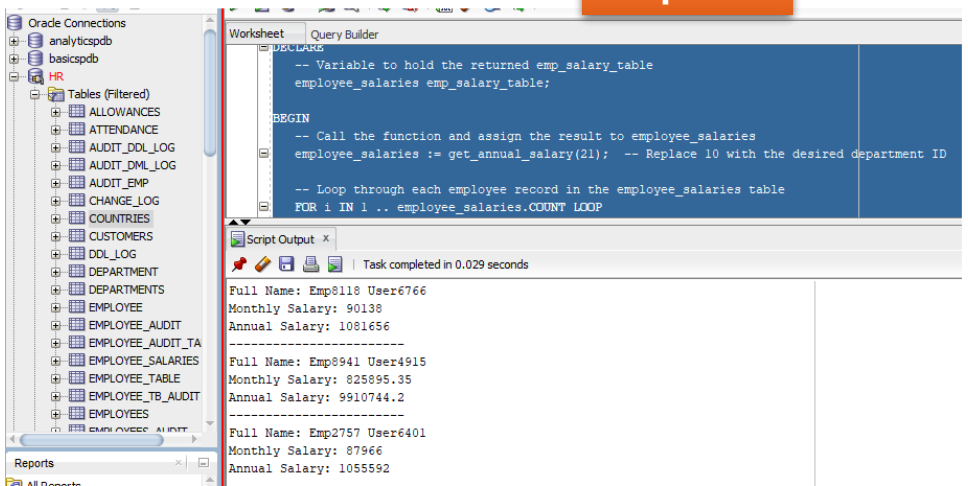
```
CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
  RETURN emp_salary_table
IS
  -- Declare a variable of the table type to store results
  emp_salaries emp_salary_table := emp_salary_table();
  -- Declare an explicit cursor for fetching employee data
  CURSOR emp_cursor IS
    SELECT first_name || ' ' || last_name AS full_name,
           salary AS monthly_salary,
           salary * 12 AS annual_salary
    FROM employees
    WHERE department_id = p_department_id;

  -- Variables to store individual column values
  v_full_name       VARCHAR2(100);
  v_monthly_salary  NUMBER;
  v_annual_salary   NUMBER;
BEGIN
  -- Open the cursor
  OPEN emp_cursor;
  -- Fetch each row from the cursor and populate the emp_salaries table
  LOOP
    FETCH emp_cursor INTO v_full_name, v_monthly_salary, v_annual_salary;
    -- Exit when there are no more rows to fetch
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Add each fetched row as an object to the emp_salaries table
    emp_salaries.EXTEND;
    emp_salaries(emp_salaries.COUNT) := emp_salary_obj(v_full_name, v_monthly_salary, v_annual_salary);
  END LOOP;
  -- Close the cursor
  CLOSE emp_cursor;
  -- Return the populated emp_salaries table
  RETURN emp_salaries;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
    RETURN emp_salary_table();  -- Return an empty table if no data is found
  WHEN INVALID_CURSOR THEN
    DBMS_OUTPUT.PUT_LINE('Error: Attempted to operate on an unopened or closed cursor.');
    RETURN emp_salary_table();  -- Return an empty table if there's an invalid cursor operation
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    RETURN emp_salary_table();  -- Return an empty table for any other errors
END;
/
```

Call the function

Call the function

```
DECLARE
  -- Variable to hold the returned emp_salary_table
  employee_salaries emp_salary_table;

BEGIN
  -- Call the function and assign the result to employee_salaries
  employee_salaries := get_annual_salary(21);  -- Replace 10 with the desired department ID

  -- Loop through each employee record in the employee_salaries table
  FOR i IN 1 .. employee_salaries.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Full Name: ' || employee_salaries(i).full_name);
    DBMS_OUTPUT.PUT_LINE('Monthly Salary: ' || employee_salaries(i).monthly_salary);
    DBMS_OUTPUT.PUT_LINE('Annual Salary: ' || employee_salaries(i).annual_salary);
    DBMS_OUTPUT.PUT_LINE('-----------------------');
  END LOOP;

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred while retrieving salaries: ' || SQLERRM);
END;
/
```

Output



Question – About Using cursor in Package

You are tasked with writing a PL/SQL package to process employee data for a given department. The package includes a procedure that fetches employee information (*Employee ID, First Name, Last Name, and Salary*) from the EMPLOYEES table based on the department ID.

Your procedure uses an explicit cursor and handles exceptions like **NO_DATA_FOUND** and unexpected errors (OTHERS). The following behavior is expected:

1. The procedure should output the employee's details for each record fetched from the cursor.
2. If no employees are found in the specified department, a custom exception should be raised with an appropriate error message.
3. The procedure must handle any unexpected errors gracefully by outputting the error message.

Task:

- Explain the role of ***v_row_count*** in the procedure and how it helps detect if no records are fetched.
- What would happen if the ***v_row_count*** is 0 after the cursor loop? How does the custom exception handling help in this case?
- Describe the significance of the **NO_DATA_FOUND** and OTHERS exceptions in your procedure.
- What would be the output if the procedure is executed with a department ID that does not have any employees?

Disadvantages of Explicit Cursors

Disadvantage	Explanation
Increased Complexity	Requires manual control over cursor lifecycle, increasing code verbosity and management efforts.
Resource Management Overhead	Consumes more system resources, especially if not closed properly.
Risk of Cursor Leaks	Forgetting to close cursors can lead to resource leaks, causing performance issues over time.
Lower Performance for Simple Operations	Less efficient than implicit cursors for simple, one-time operations.
Requires More Error Handling	Needs extra code for handling exceptions like NO_DATA_FOUND, increasing error management overhead.
Limited Scalability	May strain system resources when used heavily in high-load environments, reducing scalability.
Harder to Read and Maintain	Adds complexity to the code, making it less readable and harder to maintain, especially in large applications.

When to Use Explicit Cursors

Despite these disadvantages, explicit cursors are beneficial when:

- You need precise control over the data retrieval process (e.g., complex multi-row processing).
- You want to fetch and process data row-by-row, where each row requires specific handling.
- There's a need to work with the same set of data multiple times without re-executing the query.

In summary, while explicit cursors offer more control, they should be used judiciously to avoid the drawbacks outlined above, particularly in simple queries or high-performance environments.

Key Takeaways on PL/SQL Cursors

Implicit Cursors

- Automatically created by Oracle for **DML** (INSERT, UPDATE, DELETE) and **SELECT INTO** statements.
- No need for explicit declaration, opening, or closing.
- Accessible through predefined **cursor attributes** (SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT, SQL%ISOPEN).
- Suitable for retrieving **a single row** of data.

Explicit Cursors

- Defined and managed explicitly by the developer for handling **multiple rows**.
- Require **declaration, opening, fetching, and closing**.
- Used when more control over query execution is needed.
- Can use **LOOP structures** for iterating through result sets.

Cursor Attributes (Common for Both)

- %FOUND → Returns TRUE if a row was found, FALSE otherwise.
- %NOTFOUND → Returns TRUE if no row was found, FALSE otherwise.
- %ROWCOUNT → Returns the number of rows processed.
- %ISOPEN → Checks if the cursor is open (TRUE) or closed (FALSE).

Keep Learning!